

Relational Algebra

Relational Query Languages

- Languages for describing queries on a relational database
- Structured Query Language (SQL)
 - Predominant application query language
 - Declarative
- Relational Algebra
 - Intermediate language within DBMS
 - Procedural

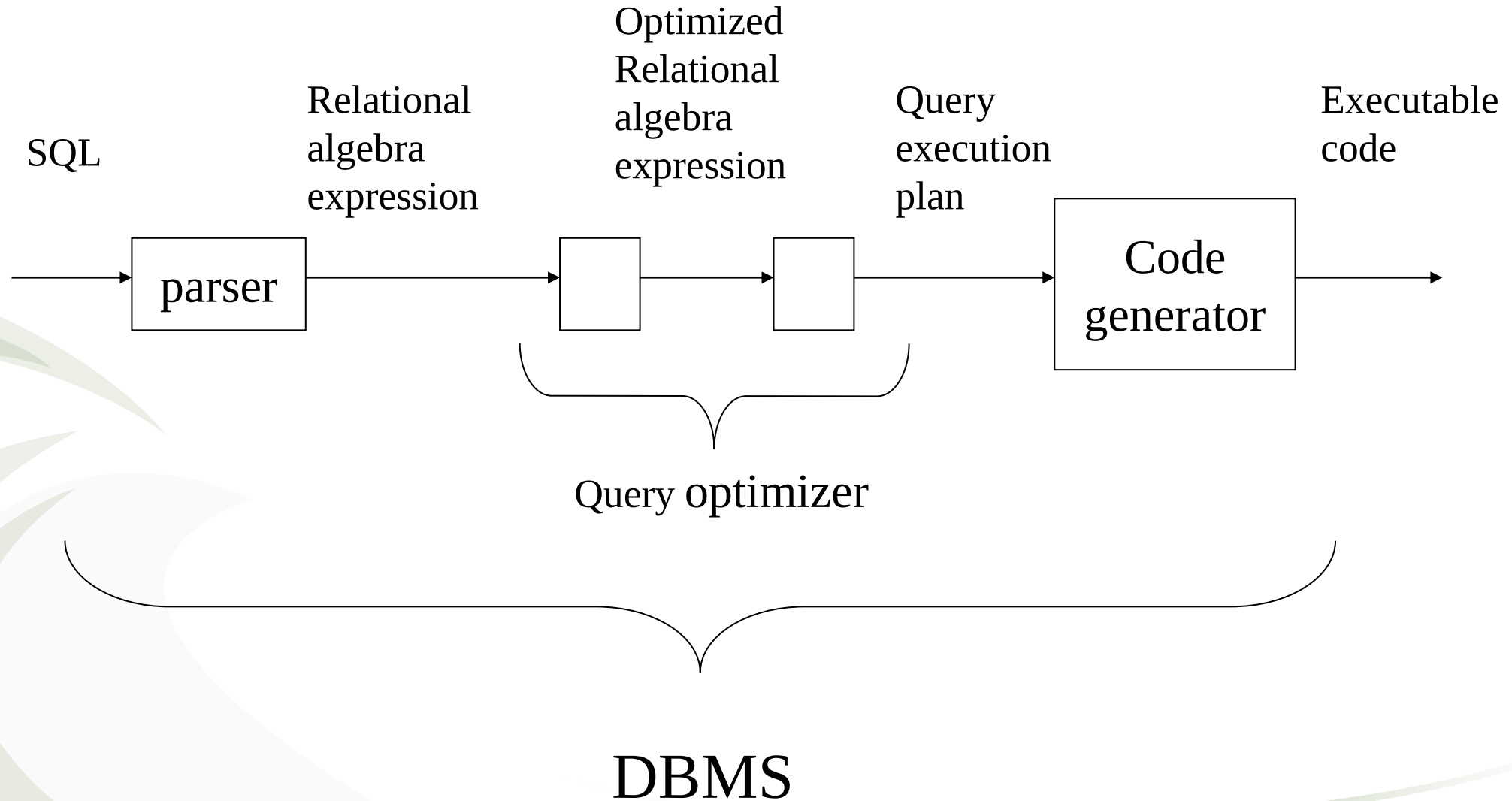
Algebra

- Based on operators and a domain of values
- Operators map arguments from domain into another domain value
- Hence, an expression involving operators and arguments produces a value in the domain. We refer to the expression as a query and the value produced as the result of that query

Relational Algebra

- Domain: set of relations
- Traditional set operators : union, intersection, difference, and cartesian product
- Special relational operators : restrict (select), project, join and divide, rename
- Procedural: Relational expression specifies query by describing an algorithm (the sequence in which operators are applied) for determining the result of an expression

Relational Algebra in a DBMS



Set Operators

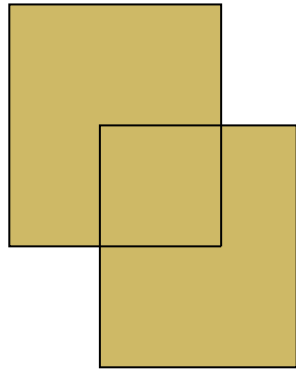
- Relation is a set of tuples => set operations should apply
- Result of combining two relations with a set operator is a relation => all its elements must be tuples having same structure
- Hence, scope of set operations limited to *union compatible relations*

Union Compatible Relations

- Two relations are union compatible if
 - Both have same number of columns
 - Names of attributes are the same in both
 - Attributes with the same name in both relations have the same domain
- Union compatible relations can be combined using union, intersection, and set difference

Union Operator

- Union



Ex. Union
(A UNION B) or
($A \cup B$)

A

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S4	Clark	20	London

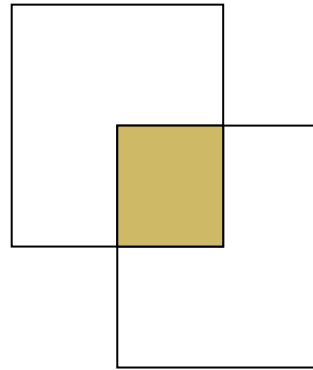
B

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S4	Clark	20	London
S2	Jones	10	Paris

Intersection Operator

- Intersection



Ex. Intersection
(A INTERSECTION B)
or $(A \cap B)$

A

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S4	Clark	20	London

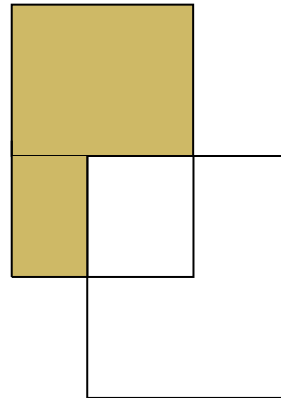
B

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris

S#	SNAME	STATUS	CITY
S1	Smith	20	London

Difference Operator

- Difference



A

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S4	Clark	20	London

B

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris

Ex. Difference (A MINUS B) or (A - B)

S#	SNAME	STATUS	CITY
S4	Clark	20	London

Ex. (B MINUS A)

Output Result ?

Cartesian Product

- If R and S are two relations, $R \times S$ is the set of all concatenated tuples $\langle x, y \rangle$, where x is a tuple in R and y is a tuple in S
 - (R and S need not be union compatible)
- $R \times S$ is expensive to compute:
 - Factor of two in the size of each row
 - Quadratic in the number of rows

a	b
x1	x2
x3	x4

R

c	d
y1	y2
y3	y4

S

a	b	c	d
x1	x2	y1	y2
x1	x2	y3	y4
x3	x4	y1	y2
x3	x4	y3	y4

$R \times S$

Select Operator

- Produce table containing subset of rows of argument table satisfying condition

$$\sigma_{\langle condition \rangle}(\langle relation \rangle)$$

- Example: $\sigma_{Hobby='stamps'}(Person)$

Id	Name	Address	Hobby
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Person

Id	Name	Address	Hobby
1123	John	123 Main	stamps
9876	Bart	5 Pine St	stamps

Selection Condition

- Operators: $<$, $\leq$, $\geq$, $>$, $=$, $\neq$
- Simple selection condition:
 - $\langle \text{attribute} \rangle \text{ operator } \langle \text{constant} \rangle$
 - $\langle \text{attribute} \rangle \text{ operator } \langle \text{attribute} \rangle$
- $\langle \text{condition} \rangle \text{ AND } \langle \text{condition} \rangle$
- $\langle \text{condition} \rangle \text{ OR } \langle \text{condition} \rangle$
- NOT $\langle \text{condition} \rangle$

Selection Condition - Examples

- $\sigma_{Id > 3000 \text{ Or } Hobby = 'hiking'}(Person)$
- $\sigma_{Id > 3000 \text{ AND } Id < 3999}(Person)$
- $\sigma_{NOT(Hobby = 'hiking')}(Person)$
- $\sigma_{Hobby \neq 'hiking'}(Person)$

Project Operator

- Produce table containing subset of columns of argument table

$\pi_{\langle \text{attribute-list} \rangle} (\langle \text{relation} \rangle)$

- Example: $\pi_{\text{Name, Hobby}} (\text{Person})$

Id	Name	Address	Hobby
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Person

Name	Hobby
John	stamps
John	coins
Mary	hiking
Bart	stamps

Project Operator

Example: $\pi_{Name, Address} (Person)$

Id	Name	Address	Hobby
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Person

Name	Address
John	123 Main
Mary	7 Lake Dr
Bart	5 Pine St

Result is a table (no duplicates)

Expressions

$\pi_{Id, Name} (\sigma_{Hobby='stamps' \text{ OR } Hobby='coins'} (Person))$

Id	Name	Address	Hobby
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Person

Id	Name
1123	John
9876	Bart

Renaming

- Result of expression evaluation is a relation
- Attributes of relation must have distinct names. This is not guaranteed with Cartesian product
 - e.g., suppose in previous example $a = c$
- Renaming operator tidies this up. To assign the names $A_1, A_2, \dots, A_n$ to the attributes of the n column relation produced by *expression* use
$$\text{expression } [A_1, A_2, \dots, A_n]$$

Example

Transcript (StudId, CrsCode, Semester, Grade)

Teaching (ProfId, CrsCode, Semester)

$\pi_{StudId, CrsCode} (Transcript) [StudId, SCrsCode] \times$
 $\pi_{ProfId, CrsCode} (Teaching) [ProfId, PCrscode]$

This is a relation with 4 attributes:

StudId, SCrsCode, ProfId, PCrsCode

Derived Operation: Join

The expression :

$$\sigma_{join-condition'} (R \times S)$$

where *join-condition'* is a *conjunction* of terms:

$$A_i \text{ oper } B_i$$

in which A_i is an attribute of R , B_i is an attribute of S , and *oper* is one of $=, <, >, \geq, \neq, \leq$, is referred to as the (theta) join of R and S and denoted:

$$R \bowtie_{join-condition} S$$

Where *join-condition* and *join-condition'* are (roughly) the same ...

Join and Renaming

- **Problem:** R and S might have attributes with the same name
 - in which case the Cartesian product is not defined
- **Solution:**
 - Rename attributes prior to forming the product and use new names in *join-condition*.
 - Common attribute names are qualified with relation names in the result of the join

Theta Join – Example

Output the names of all employees that earn more than their managers.

$$\pi_{Employee.Name} (Employee \bowtie_{MngrId=Id \text{ AND } Salary > Salary} Manager)$$

The join yields a table with attributes:

Employee.Name, Employee.Id, Employee.Salary, MngrId
Manager.Name, Manager.Id, Manager.Salary

Equijoin Join - Example

Equijoin: Join condition is a conjunction of *equalities*.

$\pi_{Name, CrsCode}(Student \bowtie_{Id=StudId} \sigma_{Grade='A'}(Transcript))$

Student

Id	Name	Addr	Status
111	John		
222	Mary		
333	Bill		
444	Joe		

Transcript

StudId	CrsCode	Sem	Grade
111	CSE305	S00	B
222	CSE306	S99	A
333	CSE304	F99	A

Name	CrsCode
Mary	CSE306
Bill	CSE304

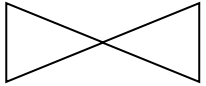
The equijoin is commonly used since it combines related data in different relations.

Natural Join

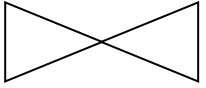
- Special case of equijoin:
 - join condition equates *all* and *only* those attributes with the same name (condition doesn't have to be explicitly stated)
 - duplicate columns eliminated from the result

Transcript (*StudId*, *CrsCode*, *Sem*, *Grade*)

Teaching (*ProfId*, *CrsCode*, *Sem*)

Transcript  *Teaching*

$\equiv \pi_{\text{StudId, Transcript.CrsCode, Transcript.Sem, Grade, ProfId}}$

(*Transcript*  *Teaching*)

[*StudId*, *CrsCode*, *Sem*, *Grade*, *ProfId*]

Natural Join (con't)

- More generally:

$$R \bowtie S = \pi_{attr-list} (\sigma_{join-cond} (R \times S))$$

where

$attr-list = attributes(R) \cup attributes(S)$
(duplicates are eliminated) and $join-cond$ has the form:

$$A_1 = A_1 \text{ AND } \dots \text{ AND } A_n = A_n$$

where

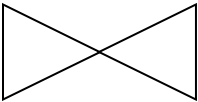
$$\{A_1 \dots A_n\} = attributes(R) \cap attributes(S)$$

Natural Join Example

- List all Id's of students who took at least two different courses:

Transcript (StudId, CrsCode, Sem, Grade)

Transcript (StudId, CrsCode2, Sem2, Grade2)

$\pi_{StudId} (\sigma_{CrsCode \neq CrsCode2}$
(Transcript 

Transcript [StudId, CrsCode2, Sem2, Grade2]))

(don't join on CrsCode, Sem, and Grade attributes)

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that do not match tuples in the other relation to the result of the join.
- Uses null values:
 - null signifies that the value is unknown or does not exist
 - All comparisons involving null are (roughly speaking) false by definition.

Outer Join Example

- Relation *instructor*

<i>ID</i>	<i>name</i>	<i>dept_name</i>
10101	Srinivasan	Comp. Sci.
12121	Wu	Finance
15151	Mozart	Music

- Relation *teaches*

<i>ID</i>	<i>course_id</i>
10101	CS-101
12121	FIN-201
76766	BIO-101

Outer Join Example

- Join

instructor ⋈ *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201

- Left Outer Join

instructor ⋈_L *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201
15151	Mozart	Music	<i>null</i>

Outer Join Example

■ Right Outer Join

instructor ⋈_r *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201
76766	null	null	BIO-101

■ Full Outer Join

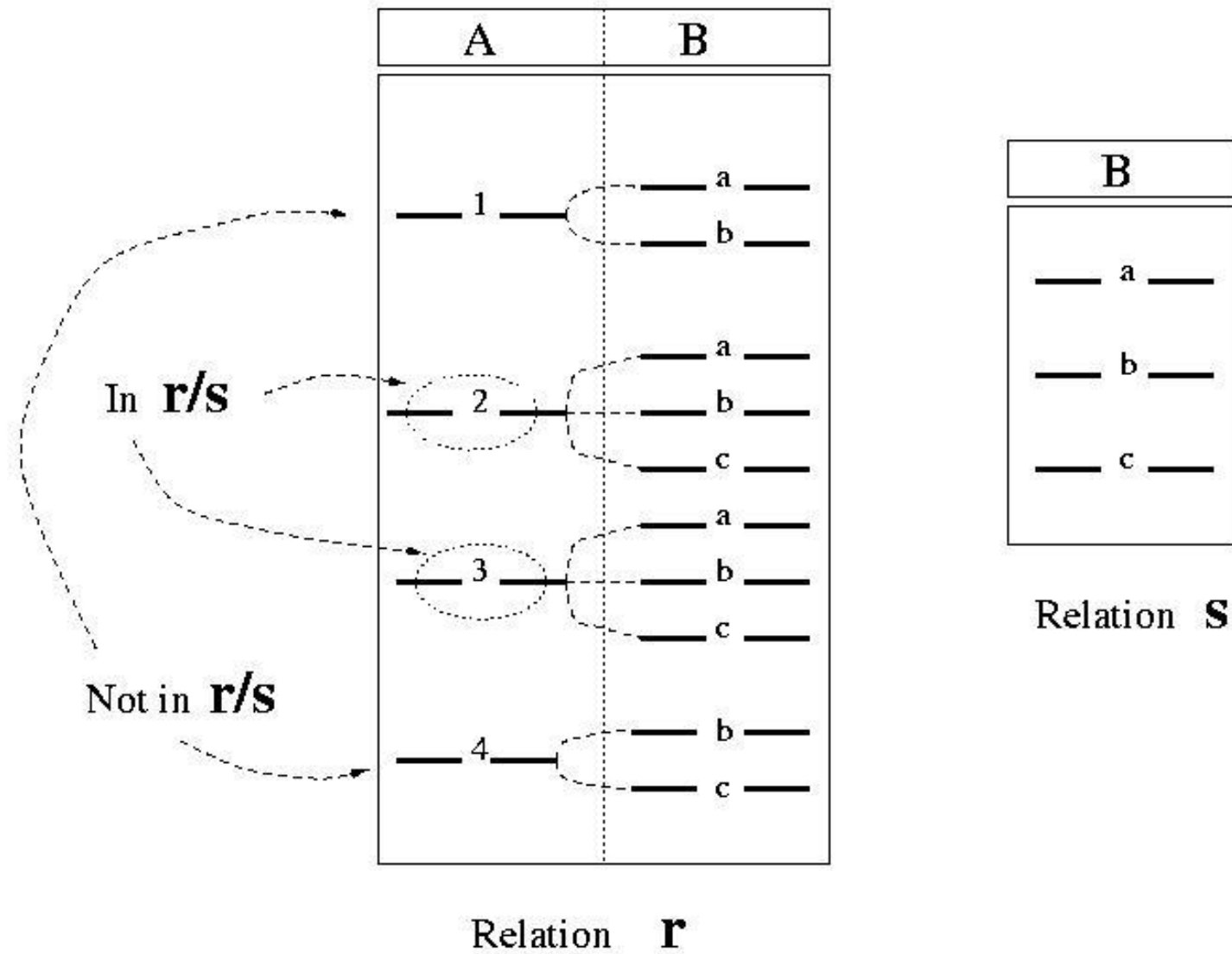
instructor ⋈_f *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201
15151	Mozart	Music	<i>null</i>
76766	null	null	BIO-101

Division

- Goal: Produce the tuples in one relation, r , that match *all* tuples in another relation, s
 - $r (A_1, \dots A_n, B_1, \dots B_m)$
 - $s (B_1 \dots B_m)$
 - r/s , with attributes $A_1, \dots A_n$, is the set of all tuples $\langle a \rangle$ such that for every tuple $\langle b \rangle$ in s , $\langle a, b \rangle$ is in r
- Can be expressed in terms of projection, set difference, and cross-product

Division (con't)



Division - Example

- List the Ids of students who have passed *all* courses that were taught in spring 2017
- *Numerator*: StudId and CrsCode for every course passed by every student

$\text{StudId, CrsCode } (\sigma_{\text{Grade} \neq 'F'} (\text{Transcript}))$

- *Denominator*: CrsCode of all courses taught in spring 2017

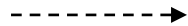
$\pi_{\text{CrsCode}} (\sigma_{\text{Semester}='S2017'} (\text{Teaching}))$

- Result is *numerator/denominator*

Division

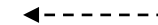
- *Query type*: Find the subset of items in one set that are related to *all* items in another set
- *Example*: Find professors who have taught courses in *all* departments
 - Why does this involve division?

Contains row
 $\langle p, d \rangle$ if professor
 p has taught a
course in
department d



ProfId	DeptId

DeptId



All department Ids

Assignment Operation

- The assignment operation ($\leftarrow$) provides a convenient way to express complex queries.
- Write query as a sequential program consisting of
 - a series of assignments
 - followed by an expression whose value is displayed as a result of the query.
- Assignment must always be made to a temporary relation variable.

Aggregate Function

- Aggregate Function takes a collection of values and returns a single value as a result.

avg : average value

min : minimum value

max : maximum value

sum : sum of values

count : number of values

Aggregate Function

- Aggregate Operation in relational algebra

$$G_1, G_2, \dots, G_n \mathcal{G} F_1(A_1), F_2(A_2), \dots, F_n(A_n) (E)$$

- E is any relational-algebra expression
 - $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (can be empty)
 - Each F_i is an aggregate function
 - Each A_i is an attribute name

Note: Some books/articles use $\mathcal{F}$ (Script Capital F/Calligraphic F) or γ (Gamma) instead of $\mathcal{G}$ (Script Capital G/Calligraphic G)

Aggregate Function Example

- Relation r

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$G_{\text{sum}(c)}(r)$

sum(c)
27

Aggregate Function Example

- Find the average salary in each department

dept_name $\overline{G}$ *avg(salary)* (*instructor*)

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

Use renaming "avg_salary" instead of function name "avg(salary)", see next slide



dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Aggregate Function Example

- Result of aggregation does not have a name
- Can use rename operation to give it a name

```
dept_name  $\mathcal{G}$  avg(salary) as avg_salary (instructor)
```